A Comparison of TCN and TimesNet for Long-Term Time Series Forecasting

Benedetto Secco[†]

***Abstract*—Time series forecasting is a fundamental task in machine learning with applications ranging from energy demand prediction to industrial monitoring. In this project, I study the problem of long-term multivariate forecasting using deep learning models and focus on comparing two different architectural paradigms: a Temporal Convolutional Network (TCN) as a strong convolutional baseline, and TimesNet, a more recent model that explicitly exploits periodic structure in time series data.**

Both models are evaluated on the Electricity dataset, a standard benchmark for multivariate forecasting. The experimental setup is kept consistent across all runs, with controlled variations in input sequence length, forecasting horizon, embedding dimension, and, for TimesNet, the number of selected dominant frequencies.

The experimental results show that TimesNet consistently outperforms the TCN baseline across most configurations, achieving lower forecasting error. This improvement is mainly attributed to its ability to decompose time series into multiple periodic components using FFT and to model them as 2D structures processed by convolutional layers.

From a practical perspective, the two approaches highlight a trade-off between accuracy and efficiency. TCN models are simpler and computationally lighter, while TimesNet instead achieves better predictive performance at the cost of additional computational overhead due to FFT-based decomposition and multi-branch convolutional processing.

Overall, the results indicate that explicitly leveraging periodic structure is highly beneficial for long-term time series forecasting.

***Index Terms*—Time Series Forecasting, Deep Learning, Temporal Convolutional Networks, TimesNet, Convolutional Neural Networks, Time Series Analysis.**

I. Introduction

Time series forecasting plays a central role in many real-world applications, including energy demand estimation, financial analysis, and infrastructure management. The ability to accurately predict future values from historical observations is crucial for decision-making processes, especially in domains where temporal dynamics exhibit complex patterns such as seasonality, trends, and noise. Among these, electricity consumption forecasting represents a particularly relevant and challenging task due to its strong periodic behavior and high variability across users.

In this report, I focus on long-term multivariate time series forecasting using the Electricity dataset [1], which consists of hourly measurements of power consumption for hundreds of clients. Modeling such data is challenging because it involves both short-term fluctuations and long-term dependencies, often intertwined across multiple periodic scales (e.g., daily or weekly patterns). Traditional approaches based on recurrent neural networks struggle with long-range dependencies and computational efficiency, while convolutional models such as Temporal Convolutional Networks (TCNs) [2] improve parallelism but still operate in a one-dimensional temporal space, limiting their ability to explicitly disentangle complex temporal structures.

To address these challenges, I investigate the effectiveness of TimesNet [3], a recent deep learning architecture that models time series through a multi-periodicity perspective. The core idea is to transform the original one-dimensional sequence into a set of two-dimensional representations based on dominant periods extracted via Fourier analysis. This allows the model to jointly capture intra-period (short-term) and inter-period (long-term) variations.

The main contributions of this report can be summarized as follows:

- I implement a simplified TimesNet architecture tailored for long-term forecasting on the Electricity dataset, focusing on the role of multi-periodicity and 2D representations.
- I implement a Temporal Convolutional Network baseline within the same experimental pipeline to ensure a fair comparison.
- I evaluate both models across multiple forecasting horizons, analyzing their performance in terms of prediction error and stability.

The remainder of this report is organized as follows. Section 2 reviews the related work. Section 3 describes the dataset and preprocessing steps. Section 4 introduces the implemented models. Section 5 details their network architectures. Section 6 presents the training pipeline and experimental setup. Section 7 discusses the results and analysis. Finally, Section 8 concludes the report and outlines possible future work.

II. Related Work

Time series forecasting has been extensively studied in the literature, ranging from classical statistical models to modern deep learning approaches. Traditional methods such as ARIMA [4] assume predefined temporal structures and rely on strong assumptions about trend and seasonality. While effective in simple scenarios, these approaches often fail to capture the complex, non-linear, and multi-scale dynamics that characterize real-world data.

To overcome these limitations, deep learning models have been widely adopted for temporal modeling. Recurrent neural

[†]Department of Physics and Astronomy, University of Padova, email: benedetto.secco.1@studenti.unipd.it

networks, including LSTM [5] architectures, model sequential dependencies through hidden state transitions, but they often struggle with long-term dependencies and suffer from limited parallelism. Convolutional approaches such as TCNs address some of these issues by using dilated convolutions to enlarge the receptive field, enabling efficient and stable training. More recently, Transformer-based models [6] have been introduced to capture long-range dependencies via attention mechanisms, achieving strong performance in many forecasting tasks. However, these methods typically operate on one-dimensional representations of time series and do not explicitly exploit the underlying periodic structure of the data.

More recent works have started to incorporate frequency-domain information and periodicity-aware mechanisms to better model temporal patterns. In particular, architectures such as Autoformer [7] and FEDformer [8] leverage them to improve forecasting performance.

III. Dataset

The experiments in this project are conducted on the Electricity dataset, a widely used benchmark for multivariate time series forecasting. The dataset originates from the UCI Electricity Load Diagrams dataset [1], which contains electricity consumption measurements for 370 clients recorded at 15-minute intervals between 2011 and 2014. Due to the presence of inactive clients and a large number of zero values, a standard preprocessing pipeline introduced by Lai et al [9] is commonly adopted in the literature to obtain a cleaner and more consistent benchmark [10].

In this preprocessing procedure, the data is first restricted to the period from 2012 to 2014, discarding earlier records with excessive sparsity. Clients with a high proportion of missing or zero values are removed, resulting in a final set of 321 time series. The temporal resolution is then reduced from 15-minute intervals to hourly measurements by aggregating consecutive observations. This produces a dataset with $T = 26{,}304$ time steps and $N = 321$ variables, which can be interpreted as a matrix in $\mathbb{R}^{T \times N}$, where each column corresponds to a client and each row to a time step.

The dataset is then normalized (zero mean and unit variance) within the data loading pipeline, computed independently for each variable. The forecasting task is formulated using a sliding window approach: given an input sequence of length L time steps, the model predicts the following H time steps. This results in input tensors of shape $[B, L, N]$ and target tensors of shape $[B, H, N]$, where B denotes the batch size. Different forecasting horizons and input lengths are considered in the experiments.

Finally, the dataset is split into training, validation, and test sets using a chronological partition. This ensures that future information is not used during training and that the evaluation reflects a realistic forecasting scenario. The split ratio follows approximately 70/10/20%. The final number of samples in each split, presented in Tab. 1, slightly deviates from these proportions due to the sliding window construction, which reduces the number of valid input-output pairs near the boundaries of each segment.

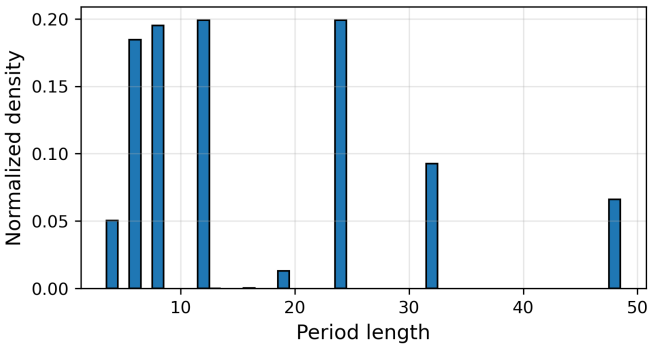


Fig. 1

Split	Number of samples
Train	18221
Validation	2535
Test	5167

TABLE 1: Chronological split of the Electricity dataset used in the experiments.

A brief analysis of the periodic structure of the dataset is performed by computing with FFT the dominant periods and selecting the first $k = 5$ for each window in the training set. The resulting distribution, shown in Fig. 1, highlights clear peaks around periods of 6, 8, 12, 24, 32, and 48 time steps. Since the data is sampled at an hourly resolution, these values correspond to meaningful temporal cycles within a day or multiples thereof (e.g., sub-daily patterns and daily periodicity). This observation confirms the strong presence of regular and interpretable temporal patterns in the dataset, which can be effectively exploited by models designed to capture periodic structure.

IV. Models

A. TimesNet

TimesNet is a deep learning architecture for time series analysis that transforms 1D temporal sequences into multi-periodic 2D representations, allowing the model to capture both intra-period and inter-period variations through 2D convolutions.

Given a hidden representation $\mathbf{X}_{\text{1D}}^{l-1} \in \mathbb{R}^{T \times d_{\text{model}}}$, where l is the layer index, T the sequence length and d_{model} the feature dimension, the model first detects dominant periodicities via FFT:

$$\mathbf{A} = \text{Avg}(\text{Amp}(\text{FFT}(\mathbf{X}_{\text{1D}}^{l-1})))$$

and selects the top-k frequencies $\{f_i\}$, with corresponding periods $p_i = \lceil T/f_i \rceil$.

For each period p_i, the sequence is zero-padded and reshaped into a 2D representation:

$$\mathbf{X}_{\text{2D}}^{l,i} \in \mathbb{R}^{p_i \times f_i \times d_{\text{model}}}$$

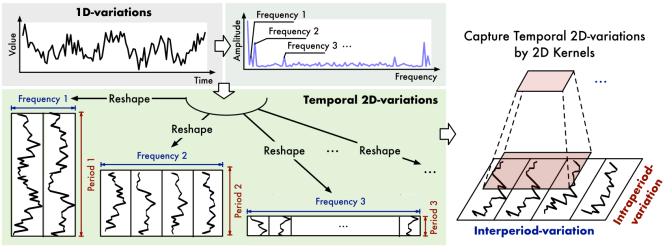


Fig. 2: TimesNet core architecture.

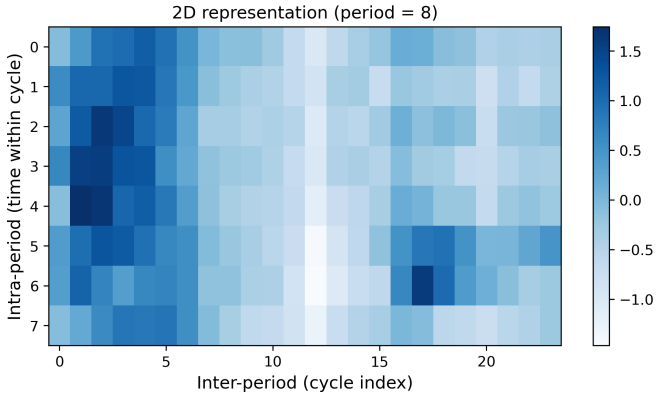


Fig. 3: Example of 2D representation obtained by rearranging a time serie of 192 time steps in 24 cycles of length 8.

which is processed by an Inception-style 2D convolutional block to capture temporal patterns. The outputs are then reshaped back to 1D sequences $\hat{\mathbf{X}}_{1D}^{l,i}$.

Finally, the k representations are combined through a weighted sum, where weights are obtained via a Softmax over the FFT amplitudes:

$$\mathbf{X}_{1D}^{l} = \sum_{i=1}^{k} \hat{\mathbf{A}}_{f_i} \hat{\mathbf{X}}_{1D}^{l,i} + \mathbf{X}_{1D}^{l-1}$$

This residual aggregation allows the model to integrate multiple periodic views while preserving the original signal.

The 2D representation reorganizes the original time series by folding it according to a detected period, producing matrices of shape $[p, f]$. In this view, each column corresponds to one period segment, while each row aligns the same phase across multiple cycles.

This representation allows one to separately analyze variations within a cycle and across cycles. In particular, a horizontal gradient, as observed in the example in Fig. 3, indicates a systematic evolution across periods: the shape of the cycle remains similar, but its overall level changes over time. This corresponds to long-term trends superimposed on the periodic structure. More generally, vertical variations describe the internal structure of a single cycle, while localized deviations highlight anomalies that do not follow the dominant periodic behavior.

B. Temporal Convolutional Networks

Temporal Convolutional Networks (TCN) model sequences using a fully convolutional 1D architecture that replaces recurrence with parallel processing. Given an input $\mathbf{X}^{l-1} \in \mathbb{R}^{T \times d_{\text{model}}}$, TCNs rely on three key mechanisms: causal convolutions, which ensure that predictions at time t depend only on past inputs; dilated convolutions, which expand the receptive field exponentially with depth,

$$\text{RF} = 1 + (k-1)(2^N - 1),$$

allowing the model to capture long-range dependencies; and residual blocks, which stabilize training through skip connections combined with nonlinearities and dropout.

V. Architecture

A. TimesNet

The implemented TimesNet architecture follows a simplified version of the original model [3], focusing on the core mechanism of periodicity-aware representation learning. The input time series $x \in \mathbb{R}^{B \times L \times N}$ is first normalized per sample (zero mean and unit variance) and projected into a latent space through a linear embedding, mapping the original N variables into a d_{model}-dimensional representation. In order to enable multi-step forecasting, the temporal dimension is expanded from input length L to $L+H$ through a linear transformation applied along the time axis.

The core of the model is a stack of TimesBlocks. Each TimesBlock takes as input a latent representation $X \in \mathbb{R}^{B \times (L+h) \times d_{\text{model}}}$ and extracts dominant periodicities using a FFT. As explained before, the top-k frequencies are selected and converted into corresponding periods, then for each period the temporal sequence is reshaped into a two-dimensional tensor of final shape $[B, d_{\text{model}}, p_i, f_i]$, where p_i represents the period length and f_i the number of repetitions. This transformation enables the model to reinterpret temporal patterns as spatial structures.

Each 2D representation is then processed by a convolutional block based on an Inception-style design. Specifically, multiple 2D convolutional kernels with different sizes are applied in parallel, allowing the model to capture patterns at different spatial scales. The outputs of these convolutions are averaged and passed through a non-linear activation (GELU), followed by a second convolutional stage that projects the features back to the original latent dimension. The processed representations are then reshaped back to the one-dimensional temporal format and truncated to remove any padding.

The outputs corresponding to the different periods are combined through an adaptive aggregation mechanism. The amplitudes obtained from the FFT are normalized via a softmax function and used as weights to compute a weighted sum of the period-specific representations. This allows the model to emphasize the most relevant periodic components while suppressing less informative ones. Finally, a residual connection is applied by adding the original input of the block, improving stability and facilitating gradient flow.

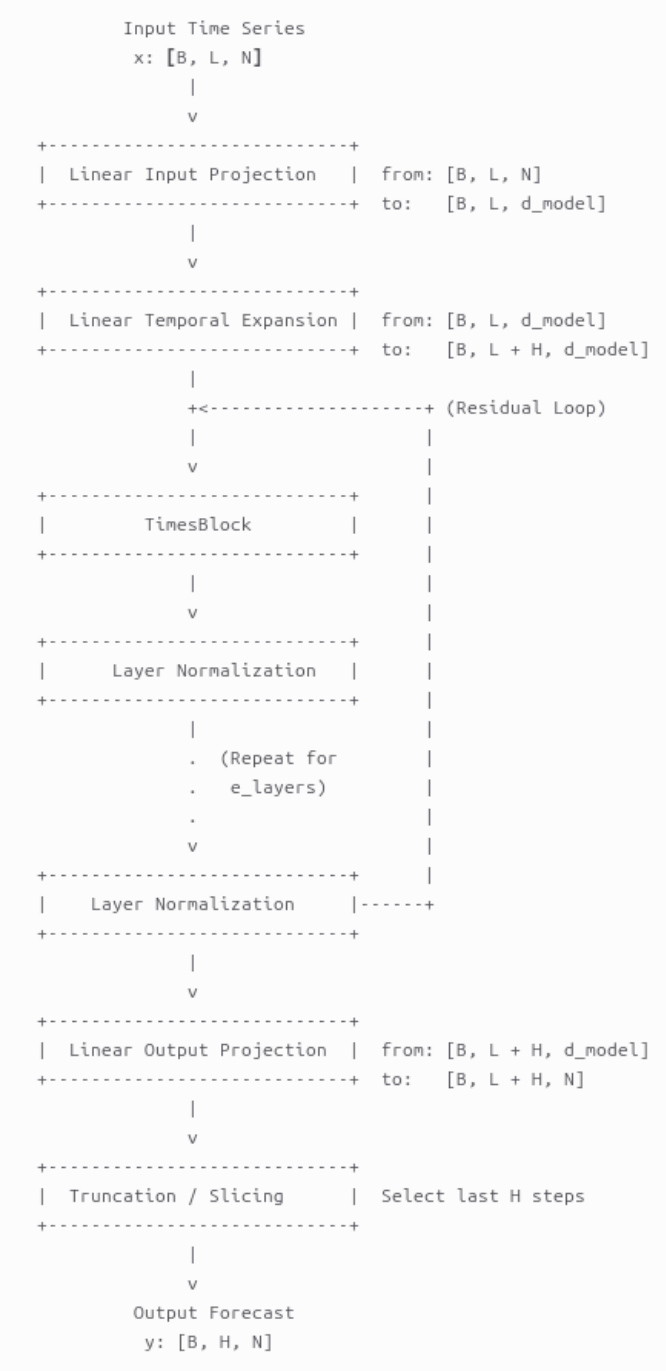


Fig. 4: The schematic of the TimesNet architecture.

The full model consists of multiple TimesBlocks stacked sequentially, each followed by layer normalization. After the final block, a linear projection maps the latent representation back to the original variable space. A final de-normalization step restores the original scale of the data, and only the last H time steps are retained as the model prediction.

B. TCN

The TCN baseline is implemented using a standard Temporal Convolutional Network architecture adapted to the forecasting setting and aligned with the design choices of TimesNet. As in the previous model, the input time series $x \in \mathbb{R}^{B \times L \times C}$ is first normalized per sample and projected into a latent space of dimension d_{model} through a linear layer. This ensures that both models operate on the same feature dimension, allowing for a fair comparison.

The core of the model is a stack of TemporalBlocks, each composed of two one-dimensional causal convolutions with exponentially increasing dilation factors. Each convolution is followed by a ReLU activation and dropout for regularization.

Within each TemporalBlock, a residual connection is employed by adding the input of the block to its output, optionally using a 1×1 convolution to match the channel dimensions. This residual structure improves gradient flow and stabilizes training, similarly to the residual connections used in TimesNet. The overall TCN backbone consists of multiple such blocks stacked sequentially, with a constant number of channels equal to d_{model}, ensuring comparable model capacity with respect to the TimesNet architecture.

After temporal feature extraction, the representation is mapped back to the original format. A linear layer is then applied along the temporal dimension to directly project the input sequence of length L into the prediction horizon of length H. Finally, an output projection maps the latent features back to the original variable space, followed by a de-normalization step to restore the original scale of the data.

This design allows the TCN to serve as a strong one-dimensional baseline, maintaining comparable embedding size, depth, and normalization strategy with respect to TimesNet, while differing fundamentally in its use of purely temporal convolutions without explicit modeling of periodic structures.

VI. Learning Pipeline

Both models are trained within the same experimental pipeline to ensure a fair and consistent comparison. The training procedure follows a standard supervised learning setup for time series forecasting, where input sequences of length L are mapped to future targets of length H. The models are optimized using the Adam optimizer with an initial learning rate of 10^{-4}, and the loss function is the Mean Squared Error (MSE), computed only on the predicted horizon. Training is performed in mini-batches, and gradients are updated through backpropagation at each iteration. The number of epochs is fixed to 10.

To improve training stability and prevent overfitting, an early stopping strategy is employed based on the validation loss. Specifically, training is interrupted if no improvement is observed for a fixed number of consecutive epochs (patience = 3), and the best model is saved and restored at the end of training. In addition, the learning rate is dynamically adjusted during training using a predefined schedule. In the main configuration adopted in this work, the learning rate is progressively reduced by a factor of 0.5 at each epoch, allowing for finer optimization in later stages.

Both models share the same data preprocessing, normalization, and batching procedures, as well as comparable architectural hyperparameters such as the embedding dimension d_{model} and the number of layers. In particular, experiments are conducted by varying the input sequence length $L \in \{96, 192\}$, the prediction horizon $H \in \{96, 192, 336\}$, and the embedding dimension $d_{\text{model}} \in \{64, 128\}$, training the models over all possible combinations of these parameters. For TimesNet, an additional set of experiments is performed to analyze the impact of the number of selected periods, with $k \in \{1, 3, 5, 7\}$ evaluated under a fixed configuration ($L = 96$, $H = 96$, $d_{\text{model}} = 128$).

All experiments are executed using the same training protocol, hardware setup, and evaluation procedure, ensuring that differences in performance can be attributed to the modeling approach rather than to discrepancies in the experimental setup.

VII. Results

The experimental results highlight consistent differences between the proposed TimesNet model and the TCN baseline across all evaluated settings. A first comparison is shown in Figure 5, where the training and validation loss curves are reported for both models over epochs. Here TimesNet achieves lower validation loss and faster convergence stability compared to the TCN. While both models successfully reduce the training error, TCN exhibits a slightly higher generalization gap, suggesting a reduced ability to capture the underlying temporal structure of the data. This behavior is consistent across all tested hyperparameter combinations.

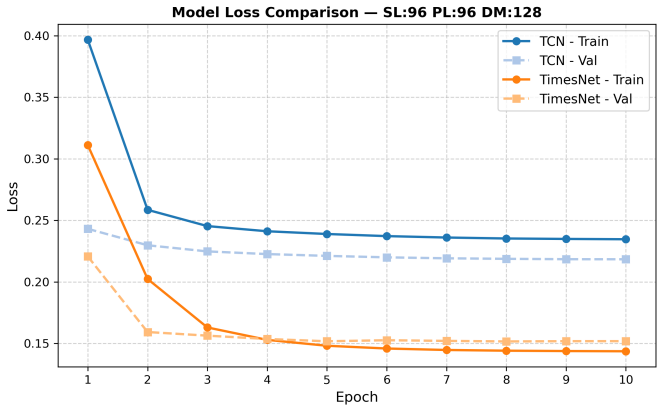


Fig. 5: MSE losses comparison between TCN and TimesNet, trained with parameters: input sequence length 96, horizon prediction length 96, model hidden dimension 128, top k equal to 5.

A more comprehensive view is provided in the 3D visualization of test losses in Figure 6, which aggregates results across all experimental configurations. The plot clearly shows that increasing the embedding dimension from 64 to 128 consistently improves performance for both models, indicating that a richer latent representation is beneficial for capturing multivariate dependencies. Similarly, longer input sequences

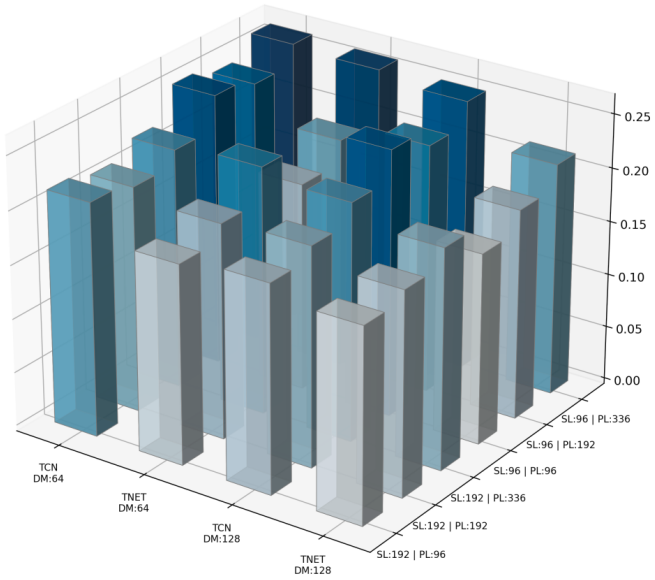


Fig. 6: MSE test losses values (on the z-axis) of the two models for each hyperparameter combination.

($L = 192$) generally outperform shorter ones ($L = 96$), confirming the importance of longer historical context for forecasting accuracy. On the other hand, increasing the prediction horizon tends to degrade performance, as expected due to the higher uncertainty in long-term forecasting. Across all configurations, TimesNet consistently achieves slightly lower test loss than TCN, confirming the advantage of explicitly modeling periodic structures. The exact test loss values are reported in Table 2.

TABLE 2: Final MSE test loss values across model configurations and sequence/prediction lengths.

	$d_{model} = 64$		$d_{model} = 128$	
(L, H)	**TCN**	**TNET**	**TCN**	**TNET**
(96, 96)	0.253	0.193	0.248	**0.177**
(96, 192)	0.245	0.214	0.231	**0.195**
(96, 336)	0.263	0.260	0.258	**0.216**
(192, 96)	0.217	0.185	0.193	**0.181**
(192, 192)	0.210	0.200	0.204	**0.189**
(192, 336)	0.223	0.230	0.221	**0.204**

Finally, qualitative results are shown in Figure 7, where representative prediction examples are compared between the two models. In the first case, both models fail to fully capture the global trend, although TimesNet remains slightly closer to the ground truth. In the second example, the TCN suffers from a noticeable vertical shift, indicating difficulty in properly aligning the predicted signal with the true scale, while TimesNet provides a more accurate reconstruction. In the third case, TimesNet is able to follow fine-grained temporal variations more closely, whereas the TCN produces a smoother and more coarse-grained approximation of the signal. Overall, these qualitative results confirm that the explicit modeling of periodic structures in TimesNet leads to more accurate and

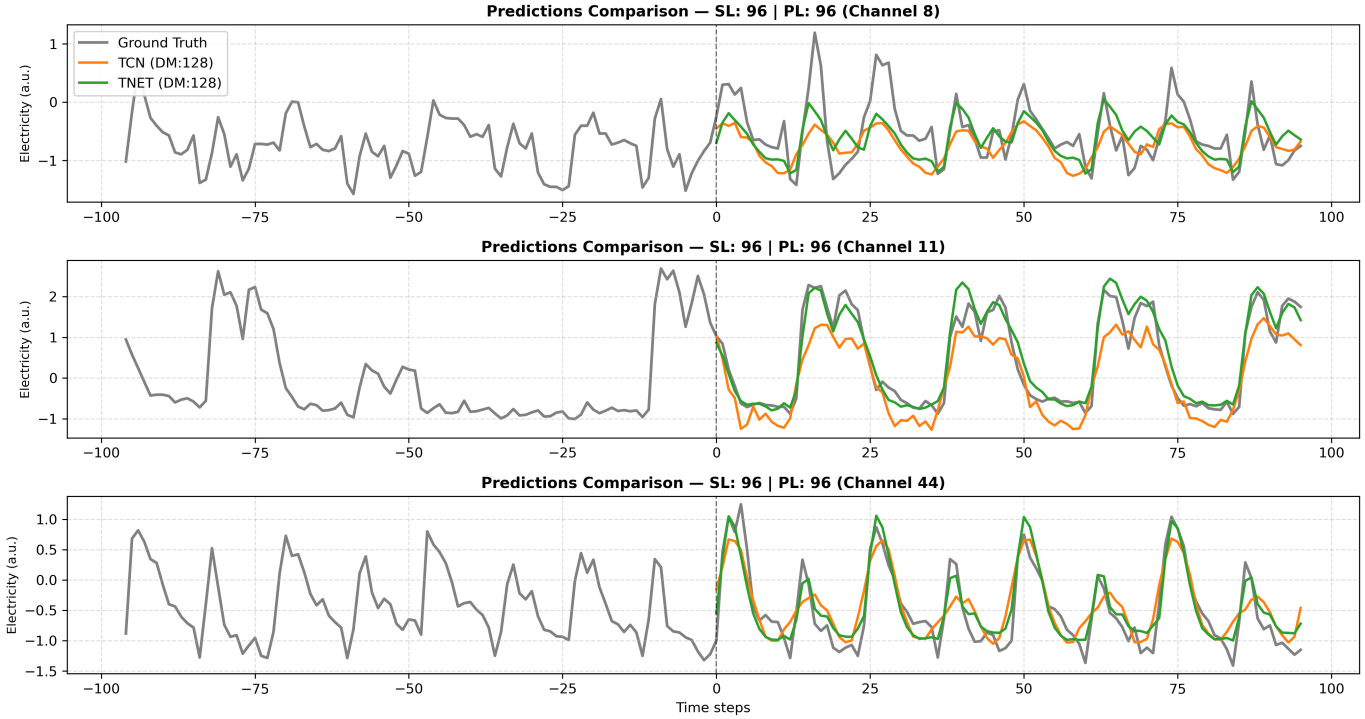


Fig. 7: Examples of predictions made with fixed parameters: sequence length 96, prediction length 96, d_{model}=128, and top-k=5

better-aligned forecasts.

For TimesNet, the influence of the hyperparameter k, which controls the number of selected dominant periods in the FFT-based decomposition, is analyzed in Fig. 8. In particular, I evaluate the model using $k \in 1, 3, 5, 7$ while keeping the configuration fixed at $L = 96$, $H = 96$, and $d_{model} = 128$.

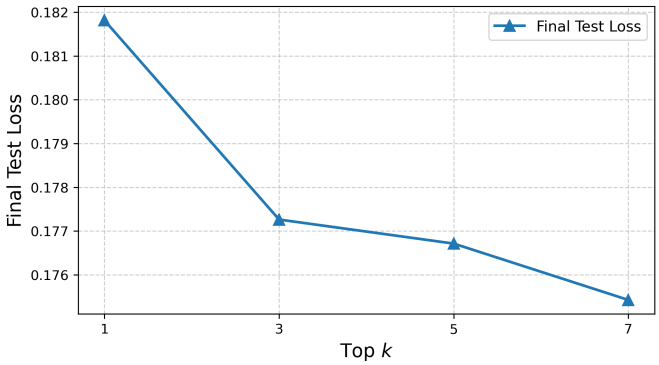


Fig. 8

The results show a consistent decrease in test loss as k increases. This behavior can be interpreted by considering the role of k in the TimesNet architecture: a larger number of retained frequencies allows the model to capture a richer set of periodic components, thus providing multiple complementary 2D representations of the same time series. This trend is consistent with the findings reported in the original TimesNet paper.

However, this improvement comes at the cost of increased computational complexity. Since each additional period requires an independent FFT-based decomposition, reshaping operation, and Inception-based convolution, the inference time grows approximately linearly with k. Therefore, there is a clear trade-off between predictive performance and efficiency: higher values of k improve accuracy but reduce computational speed. In this project, $k = 5$ represents a good balance between these two aspects and is used as the default configuration in the main experiments.

VIII. Concluding Remarks

In this project, I implemented and compared two deep learning architectures for long-term multivariate time series forecasting on the Electricity dataset: a Temporal Convolutional Network (TCN) baseline and the TimesNet model.

The results consistently show that TimesNet achieves lower forecasting error than the TCN baseline across almost all configurations. This confirms the benefit of explicitly modeling multi-periodic structure through FFT-based decomposition and 2D temporal representations. In particular, the strength of TimesNet comes from its ability to reshape temporal dependencies into 2D representations, where patterns across time can be processed using convolutional filters. At the same time, the experiments also highlight clear and intuitive trends: larger embedding dimensions improve performance, longer input contexts are generally beneficial, while excessively long prediction horizons increase uncertainty and error for both models.

From an applicability perspective, the two approaches reflect different trade-offs. TCNs are simpler, computationally efficient, and easier to deploy. TimesNet, instead, offers better predictive accuracy by explicitly leveraging periodic structure, but at the cost of higher computational complexity due to the repeated FFT decomposition and multi-branch 2D convolutions. The analysis of the k parameter further highlights this trade-off: increasing k improves performance by capturing more periodic components, but also increases runtime almost linearly.

Despite these results, several limitations remain. First, the comparison is restricted to a single dataset, and it would be valuable to evaluate both models on additional benchmarks. Second, the current implementation does not include more recent transformer-based forecasting models, which could provide a stronger reference point. Finally, future work could explore more efficient variants of TimesNet, for example by adaptively selecting k or by reducing redundant periodic representations.

Overall, the results confirm that explicitly modeling periodic structure improves forecasting performance, since it allows the model to directly exploit repeating temporal patterns that are present in the data.

REFERENCES

[1] A. Trindade, "ElectricityLoadDiagrams20112014 Data Set." https://archive.ics.uci.edu/ml/datasets/ElectricityLoadDiagrams20112014, 2015. UCI Machine Learning Repository.

[2] S. Bai, J. Z. Kolter, and V. Koltun, "An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling," *arXiv preprint arXiv:1803.01271*, Mar. 2018.

[3] H. Wu, T. Hu, Y. Liu, H. Zhou, J. Wang, and M. Long, "TimesNet: Temporal 2D-Variation Modeling for General Time Series Analysis," in *International Conference on Learning Representations*, (Kigali, Rwanda), May 2023.

[4] O. D. Anderson and M. Kendall, "Time-series. 2nd edn," *Journal of the Royal Statistical Society: Series D (The Statistician)*, vol. 25, p. 308, Dec. 1976.

[5] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, pp. 1735–1780, Nov. 1997.

[6] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, and W. Zhang, "Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, pp. 11106–11115, May 2021.

[7] H. Wu, J. Xu, J. Wang, and M. Long, "Autoformer: Decomposition Transformers with Auto-Correlation for Long-Term Series Forecasting," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, pp. 22419–22430, Dec. 2021.

[8] T. Zhou, Z. Ma, Q. Wen, X. Wang, L. Sun, and R. Jin, "FEDformer: Frequency enhanced decomposed transformer for long-term series forecasting," in *International Conference on Machine Learning*, pp. 27268–27286, July 2022.

[9] G. Lai, W.-C. Chang, Y. Yang, and H. Liu, "Modeling Long- and Short-Term Temporal Patterns with Deep Neural Networks," in *The 41st International ACM SIGIR Conference on Research & Development in Information Retrieval*, pp. 95–104, July 2018.

[10] G. Lai, "Multivariate Time Series Data - Electricity." https://github.com/laiguokun/multivariate-time-series-data/tree/master/electricity, 2017. GitHub repository.